

Theory of Computation

Regular Grammar

- * Noam chomsky gave a mathematical model & grammar which is effective for writing computer language.
- * The four types of grammar according to noam chomsky are

Grammar Type	Grammar Accepted	Language Accepted	Automation
Type-0	Unrestricted Grammar	Recursively Enumerable Language	Turing Machine
Type-1	Context Sensitive Grammar	Context Sensitive Language	Linear Bounded Automation
Type-2	Context Free Grammar	Context Free Language	Pushdown Automation
Type-3	Regular Grammar	Regular Language	Finite State Automation

Grammar: A grammar 'G' can be formally described using 4 tuples as: $G = \{V, T, S, P\}$ where,

V = Set of variables or Non-Terminal Symbols
 T = Set of Terminal Symbols

S = Start Symbol

P = Production rules for terminal & non-terminal symbols

* What is production rules?

* Ex: A production rule has the form $\alpha \rightarrow \beta$, where α and β are strings on $(V \cup T)^*$ and at least one symbol of α belongs to V .

* Ex: $G = \{ \underbrace{\{S, A, B\}}_V, \underbrace{\{a, b\}}_T, \underbrace{S}_S, \underbrace{\{S \rightarrow AB, A \rightarrow a, B \rightarrow b\}}_P \}$

* $G = \{ \{S, A, B\}, \{a, b\}, S, \{S \rightarrow AB, A \rightarrow a, B \rightarrow b\} \}$ derive

$S \rightarrow AB$

$\rightarrow ab$

$\therefore \begin{cases} A \rightarrow a \\ B \rightarrow b \end{cases}$

* Two types of Regular Grammar

Left Linear Grammar

1. A grammar is said to be left linear if all production rules of the form

$A \rightarrow XB$

$A \rightarrow X$

where, $A, B \in V$ and $X \in T$

Right Linear Grammar

1. A grammar is said to be right linear if all the production rules of the form

$A \rightarrow XB$

$A \rightarrow X$

where, $A, B \in V$ and $X \in T$

$$\underline{\text{Ex:}} \quad S \rightarrow S b b l b$$

$$\underline{\text{Ex:}} \quad S \rightarrow a b S l b$$

* Derivation from a grammar:

The set of all strings, that can be derived from a grammar is said to be the language generated from that grammar.

* Ex: Consider the grammar, $G_1 = \{\{S, A\}, \{a, b\}, S, \{S \rightarrow a A b, a A \rightarrow a a A b, A \rightarrow \epsilon\}\}$

Solⁿ: $S \rightarrow a A b$

$$\rightarrow a a A b b \text{ [by } a A \rightarrow a a A b]$$

$$\rightarrow a a a A b b b \text{ [by } a A \rightarrow a a A b]$$

$$\rightarrow a a a b b b \text{ [by } A \rightarrow \epsilon]$$

Date 27.03.2022
Day: Sunday.

Context Free Language (CFL)

In formal language theory, a CFL is a language generated by some CFG.

The set of all CFL is identical to the set of language accepted by Pushdown Automata.

CFG is denoted by 4 tuples, as, $G = \{V, \Sigma, S, P\}$ where

V = Set of variables of non-terminal symbol,

Σ = Set of Terminal Symbols

S = Start symbol

P = Production rule

CFG is the production rule of the form:

Where, $\alpha \in \{V \cup \Sigma\}^*$

and $A \in V$

CFL

Ex For generating a language that generates equal no of a's and b's in the form $a^n b^n$, the CFG will be der

$$\text{as, } G = \{ \underbrace{\{S, A\}}_{\Sigma}, \underbrace{\{a, b\}}_{\Sigma}, \underbrace{\{S \rightarrow aAb, A \rightarrow aAb | \epsilon\}}_{P} \}$$

Soln

$$S \rightarrow aAb$$

$$\rightarrow aaAbb \text{ [by, } A \rightarrow aAb]$$

$$\rightarrow aaaAbbb \text{ [by, } A \rightarrow aAb]$$

$$\rightarrow aaabbb \text{ [} A \rightarrow \epsilon]$$

$$\rightarrow a^3b^3 \text{ [a \& b are always same power]}$$

" " " " power " "

Method to find whether a string belongs to a Grammar or not.

1) Start with the start symbol and choose the closest production that matches to the given string

2) Replace the variable with its most appropriate production
Repeat the process until the string is generated
until no other production are left.

3) Verify whether the grammar, $S \rightarrow \overset{\text{Zero}}{0}B \mid 1A,$

$A \rightarrow \underline{0} | \underline{0S} | \underline{1AA} | ^\wedge, B \rightarrow \underline{1} | \underline{1S} | \underline{0BB}$

generates the string, 00110101

$S \rightarrow 0B$

$\rightarrow 00BB \quad [B = 0BB]$

$\rightarrow 001B \quad [B = 1]$

$\rightarrow 0011S \quad [B = 1S]$

$\rightarrow 00110B \quad [S = 0B]$

$\rightarrow 001101S \quad [B = 1S]$

$\rightarrow 0011010B \quad [S = 0B]$

$\rightarrow 00110101 \quad [B = 1]$

OR,

$S \rightarrow 0B$

$\rightarrow 00BB \quad [B = 0BB]$

$\rightarrow 001SB \quad [B = 1S]$

$\rightarrow 0011AB \quad [S = 1A]$

$\rightarrow 00110SB \quad [A = 0S]$

$\rightarrow 001101AB \quad [S = 1A]$

$\rightarrow 00110101 \quad [B = 1; A = 0]$

*Ex

Verify whether the grammar, $S \rightarrow aAb$, $A \rightarrow aAb \mid \Lambda$
generate the string $aabbb$

$$\begin{aligned} S &\rightarrow aAb \\ &\rightarrow aaAbb [A \rightarrow aAb] \\ &\rightarrow aabb [A \rightarrow \Lambda] \end{aligned}$$

Derivation Tree

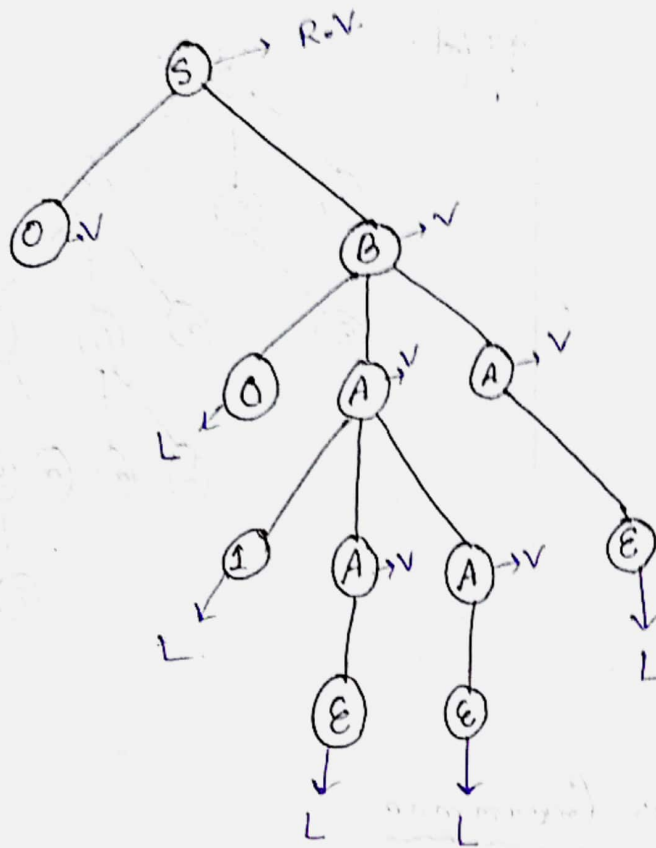
* A derivation tree or parse tree is an ordered rooted tree that graphically represents the semantic information of strings from a CFG.

Ex For the Grammar, $G = \{V, T, P, S\}$ where $S \rightarrow aB$,
 $A \rightarrow \Lambda \mid AA$, $B \rightarrow aAA$.

Root vertex: must be labelled by the start symbol.

Vertex: Labelled by root terminal

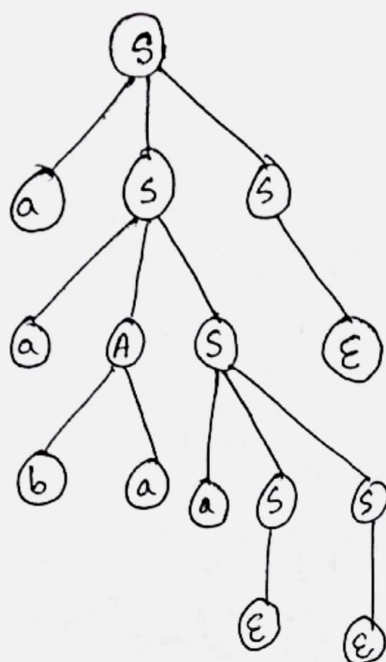
Leaves: " " Terminal symbol on ϵ



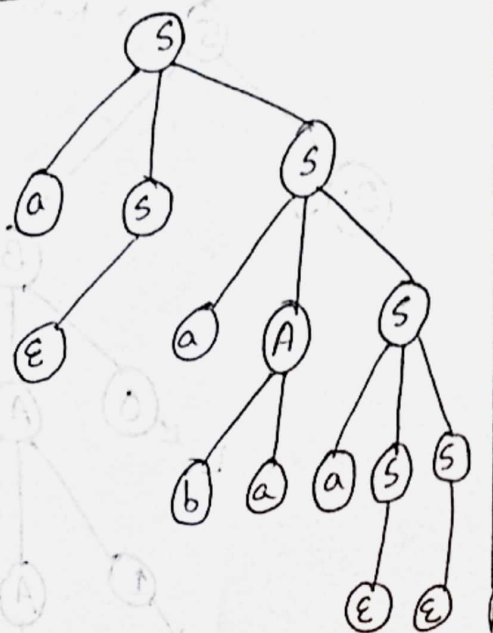
* Types of Derivation Tree → Two types: ① Left derivation tree
② Right " " "

Left Derivation Tree	Right Derivation Tree
1. Left derivation tree is obtained by applying production to the leftmost variable at in each step.	1. Right derivation tree is obtained by applying production to the rightmost variable in each step.
<u>Ex.</u> For generating the string <u>aabaa</u> from the grammar $S \rightarrow aAS aSS \epsilon, A \rightarrow Sba ba$	<u>Ex.</u> For generating the string <u>aabaa</u> from the grammar, $S \rightarrow aAS aSS \epsilon, A \rightarrow Sba ba$

Left



Right:



Ambiguous Grammar

A grammar is said to be ambiguous if there exists two or more derivation tree from for a string w (that means two or more left derivation trees)

Ex: $G = \{ \{S\}, \{a+b, +, *\}, P, S \}$ where P consists of

$$S \rightarrow S+S | S*S | a | b$$

The string $(a+a*b)$ can be generated as:

$ \begin{aligned} S &\rightarrow S+S \\ &\rightarrow a+S \\ &\rightarrow a+S*S \end{aligned} $	$ \begin{aligned} &\rightarrow a+a*S \\ &\rightarrow a+a*b \end{aligned} $	$ \begin{aligned} S &\rightarrow S+S && \rightarrow a+a*b \\ &\rightarrow S+S*S \\ &\rightarrow a+S*S \\ &\rightarrow a+a*S \end{aligned} $
--	---	--

Simplification of CFGs

06.03.22

In CFGs, sometimes all the production rules and symbols are not needed for the derivation of strings. Besides this, there may also be some ~~NULL~~^{NULL} production and unit production. Elimination of productions and symbols is called reduction of CFGs. Simplification consists these steps:

- ① Reduction of CFGs; ② Removal of unit production
- ③ Removal of NULL production.

❏ Removal of Unit Production:

Any production rule of the form $A \rightarrow B$, where $A, B \in \text{Non terminals}$ is called unit production.

* Procedure for Removal:

- ① To remove $A \rightarrow B$, add production $A \rightarrow x$ to the grammar rule wherever $B \rightarrow x$ occurs in the grammar $x \in \text{Terminal}$, x can be NULL
- ② Delete $A \rightarrow B$ from the Grammar
- ③ Repeat the steps until production are removed.

Ex. 2

Remove unit production from the grammar whose production rule is given by, $P: S \rightarrow XY, X \rightarrow a, Y \rightarrow Z|b$

$Z \rightarrow M, M \rightarrow N, N \rightarrow a.$

Solⁿ:

$Y \rightarrow Z, Z \rightarrow M, M \rightarrow N.$

① Since, $N \rightarrow a$, we add $M \rightarrow a$

$P: S \rightarrow XY, X \rightarrow a, Y \rightarrow Z|b, Z \rightarrow M, M \rightarrow a, N \rightarrow a$

② Since, $M \rightarrow a$, we add $Z \rightarrow a$

$P: S \rightarrow XY, X \rightarrow a, Y \rightarrow Z|b, Z \rightarrow a, M \rightarrow a, N \rightarrow a$

③ Since, $Z \rightarrow a$, we add $Y \rightarrow a$

$P: S \rightarrow XY, X \rightarrow a, Y \rightarrow a|b, Z \rightarrow a, M \rightarrow a, N \rightarrow a$

Remove the unreachable symbols,

$P: S \rightarrow XY, X \rightarrow a, Y \rightarrow a|b$

Removal of NULL Production:

In a CFG, a Non-terminal symbol 'A' is a nullable variable if there is a production $A \rightarrow \epsilon$ or there is a derivation that starts at 'A' and leads to ϵ (Like $A \rightarrow \dots \epsilon$).

* Procedure for Removal:

- ① To remove, $A \rightarrow \epsilon$, look for all production rules whose right side contains 'A'.
- ② Replace each occurrence of 'A' in each of these production with ' ϵ '
- ③ Add the resultant production to the grammar.

Ex

Remove NULL production from the grammar :-

$S \rightarrow ABAC, A \rightarrow aA|\epsilon, B \rightarrow bB|\epsilon, C \rightarrow c$

Solⁿ: $A \rightarrow \epsilon, B \rightarrow \epsilon$.

① To eliminate, $A \rightarrow \epsilon$

$S \rightarrow ABAC$

$S \rightarrow ABC|BAC|BC$

$A \rightarrow aA$

$\rightarrow a$

New production:

$S \rightarrow ABAC|ABCBAC|BC$

$A \rightarrow aA|a, B \rightarrow bB|\epsilon, C \rightarrow c$

② To eliminate $B \rightarrow \epsilon$

$$S \rightarrow AAc | A c | c$$

$$A \rightarrow b$$

Production Rules:

$$S \rightarrow ABAC | ABC | BC | AAC | AC | C.$$

$$A \rightarrow aA | a$$

$$B \rightarrow bB | a$$

New production: ~~rules~~

$$S \rightarrow ABAC | ABC | BAC | BC$$

$$A \rightarrow aA | a, B \rightarrow bB | \epsilon, C \rightarrow c$$

Chomsky's Normal Form (CNF)

CNF stands for Chomsky's Normal Form. A CFG is in CNF if all production rules satisfy one of the following conditions:

① ~~Third~~ Symbol Generation ϵ .
Start - Ex. $A \rightarrow \epsilon$

② A non-terminal generating two non-terminals.

Ex. $S \rightarrow AB$

③ A non-terminal generating a terminal.

Ex. $S \rightarrow a$

Ex: $G_1 = \{ S \rightarrow AB$
 $S \rightarrow C$
 $A \rightarrow a$
 $B \rightarrow b \}$

$\Rightarrow G_1$ satisfy the rules specified for CNF. So it is in CNF.

$G_2 = \{ S \rightarrow aA,$
 $A \rightarrow a,$
 $B \rightarrow c \}$

$\Rightarrow G_2$ does not satisfy the rules specified for CNF as $S \rightarrow aA$ contains terminal symbol followed by Non-terminal symbols. So, G_2 is not in CNF.

④ Steps of converting CFG into CNF:

* Step 1 - Eliminate start symbol from the RHS. If the start symbol T is at the right hand side of any production. create a production as:- $S_1 \rightarrow S$ where, S_1 is the new start symbol.

* Step 2: In the grammar, remove the null, unit and useless productions.

* Step 3: Eliminate terminals from the right hand side of the production if they exist with other non-terminals or terminals.

Ex $S \rightarrow aA$ can be decomposed as -

$$S \rightarrow RA$$

$$R \rightarrow a$$

* Step 4: Eliminate right hand side with more than two non-terminals.

Ex $S \rightarrow ABS$ can be decomposed as

$$S \rightarrow RS \quad [\text{let } R = AB]$$

$$R \rightarrow AB$$

Theory of Computation

Date: 23.04.22

Reduction of CFG

☐ Phase 1: Derivation of an equivalent grammar G' , from the CFG such that each variable derives some terminal string.

* Derivation Procedure -

- Step 1: Include all symbols W_1 , that derives some terminal and initialize $i=1$.
- Step 2: Include symbol W_{i+1} that derives W_i .
- Step 3: Increment i and repeat step 2 until $W_{i+1} = W_i$.
- Step 4: Include all procedure rules that have W_i in it.

Example: Find a reduced grammar equivalent to the grammar G , having production rules $P: S \rightarrow AC \mid B, A \rightarrow a, C \rightarrow c \mid BC, E \rightarrow aA \mid e$

Solution: Phase 1
 $T = \{a, c, e\}$
 $W_1 = \{A, C, E\}$

$$W_2 = \{A, C, E, S\}$$

$$W_3 = \{A, C, E, S\}$$

$$G' = \{(A, C, E, S), (a, c, e), P, (S)\}$$

$$P: S \rightarrow AC, A \rightarrow a, C \rightarrow c, E \rightarrow aA|e$$

Phase 2: Derivation of an equivalent grammar G'' , from the CFG, G' , such that each symbol appears in a sentential form.

* Derivation Procedure:

- Step 1: Include the start symbol y_1 and initialize $i=1$
- Step 2: Include all symbol y_{i+1} , that can be derived from y_i and include all production rules that have been applied.
- Step 3: Increment i and repeat step 2, until $y_{i+1} = y_i$

Example Phase 2

$$y_1 = \{S\}$$

$$y_2 = \{S, A, C\}$$

$$y_3 = \{S, A, C, a, c\}$$

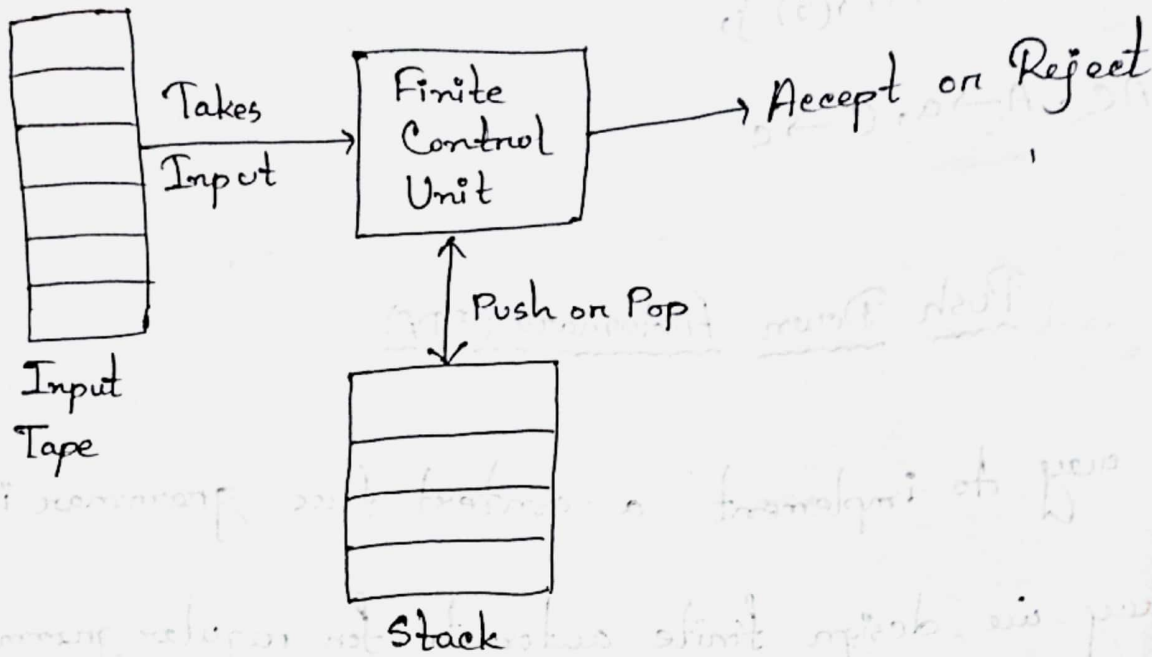
$$y_4 = \{S, A, C, a, c\}$$

$$G' = \{ (A, C, S), (a, c), P, (S) \},$$

$$P: S \rightarrow AC, A \rightarrow a, C \rightarrow c$$

Push Down Automata (PDA)

- A PDA is a way to implement a context free grammar in a similar way we design finite automata for regular grammar
- It is powerful than finite state machine (FSM)
- FSM has a very limited memory but PDA has more memory.
- PDA = FSM + Stack
- * A stack is a way we arrange element one on top of another
- * A stack does two basic operations.
 - ① Push → A new element is added at the top of the stack
 - ② Pop → The top element of the stack is read and removed.
- * Push Down Automata has 3 components:
 - ① An input tape
 - ② A finite control unit
 - ③ A stack with infinite size.



Formal Definition of Push Down Automata

A PDA is formally defined by 7 Tuples, as shown below:

$$P = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$$

where, Q = A finite set of states

Σ = A finite set of input symbols

Γ = A finite set of input ^{stack} symbols alphabet

δ = The transition function

q_0 = The start state

z_0 = The start state ~~symbol~~ stack symbol

F = The set of final / Accepting states

* δ , the transition function takes as argument a triples
 (q, a, x) where:-

- ① q is in state Q
- ② a is either a input symbol in Σ , or, $a = \epsilon$
- ③ x is a stack symbol, that is a member of Γ .

* The output of δ is finite set of pairs (P, Y) where:-

P is a new state

Y is a string of stack symbols that replaces x at the top of stack.

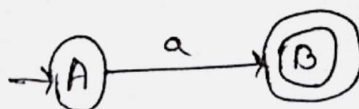
Ex: If $Y = \epsilon$ then the stack is popped.

if $Y = x$ then the stack is unchanged

if $Y = YZ$ then x is replaced by z and Y is pushed onto the stack.

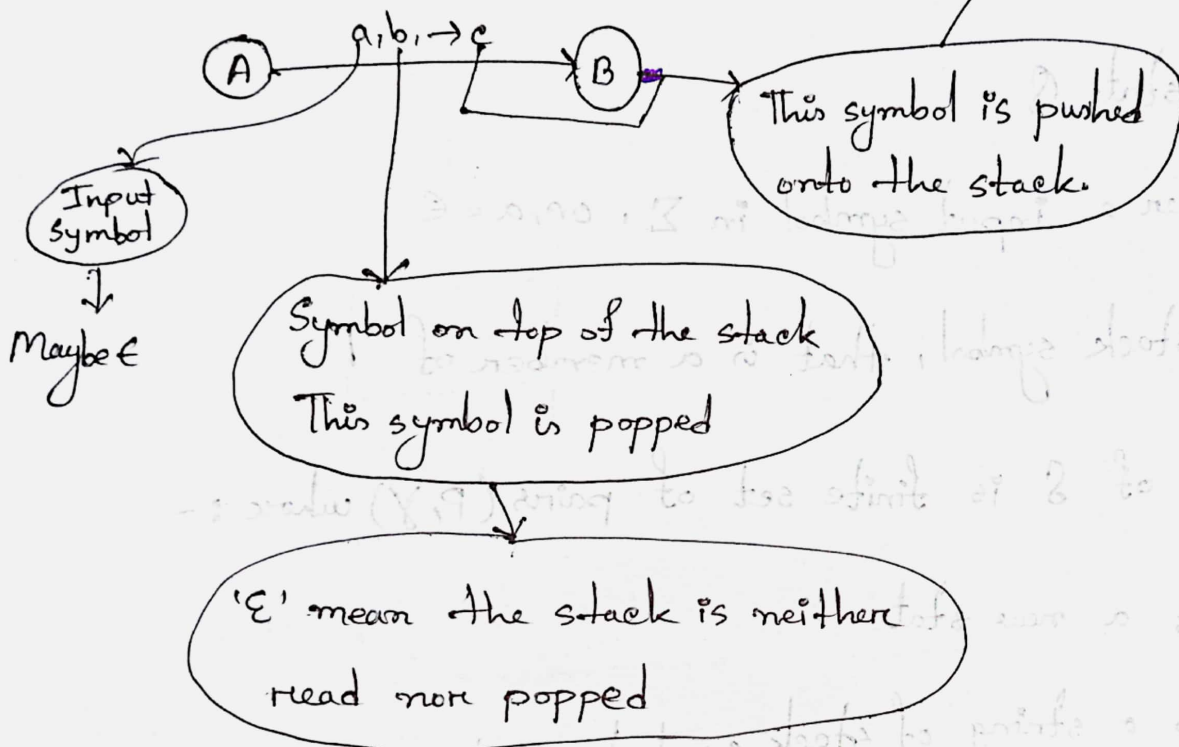
▣ Graphical Notation of PDA:

Finite State Machine:



Push Down Automata:

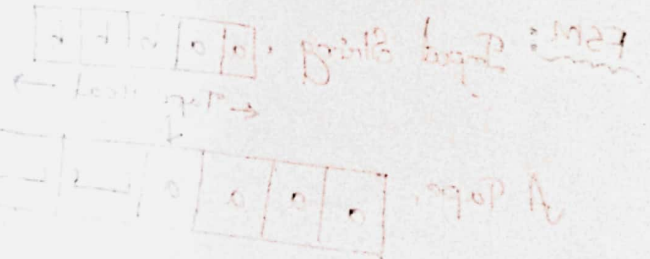
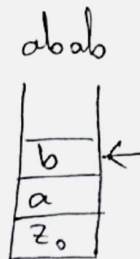
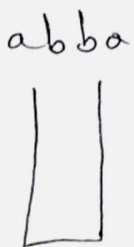
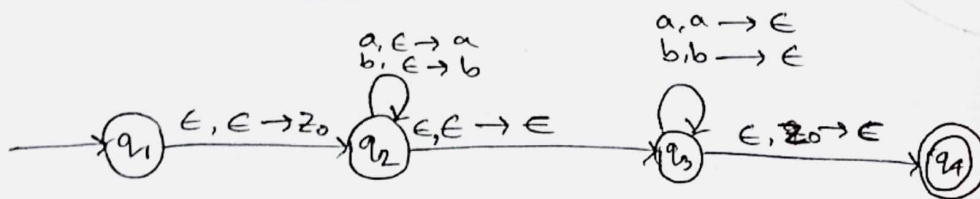
'ε' means nothing is pushed



* Construct a PDA that accepts Even Palindromes of the form
 $L = \{ww^R \mid w = (a+b)^*\}$

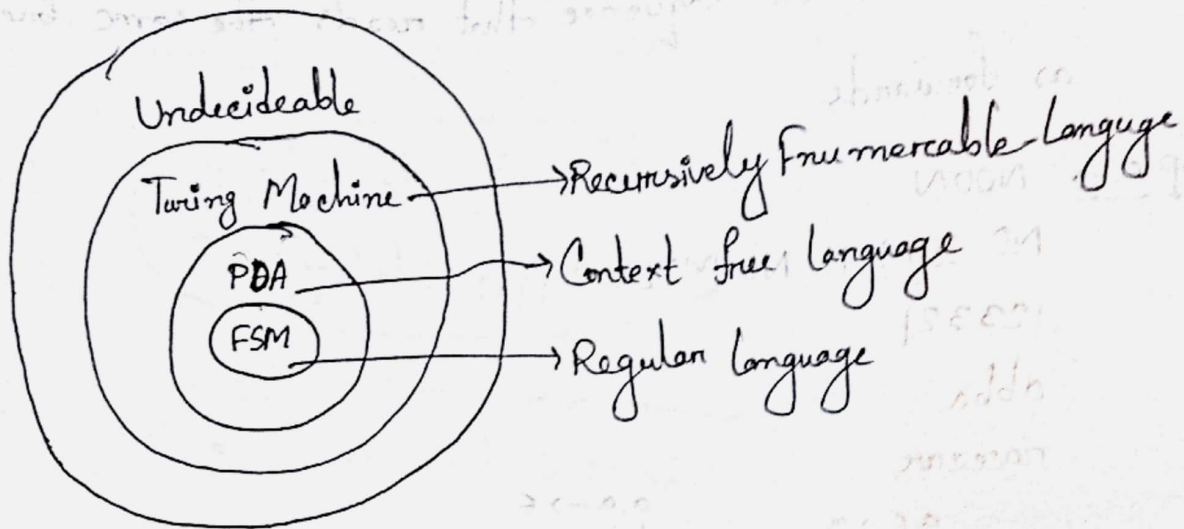
• Palindromes: A word or sequence that reads the same backwards as forwards

Example → NOON
 NO LEMON NO MELON
 123321
 abba
 racecar



Handwritten notes and diagrams at the bottom of the page, including a small state transition diagram with a state labeled '100' and various scribbles and text.

Turing Machine



FSM: Input String :

a	a	b	b	b
---	---	---	---	---

← Tape Head →

A Tape,

a	a	a	a					
---	---	---	---	--	--	--	--	--

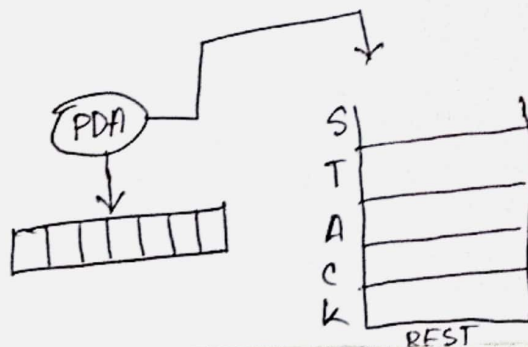
The Alphabets: $\Sigma = \{0, 1, a, b, x, z_0\}$

- * The blank is a special symbol ($\sqcup \notin \Sigma$)

- * The blank is a special symbol used to fill in the infinite tape.

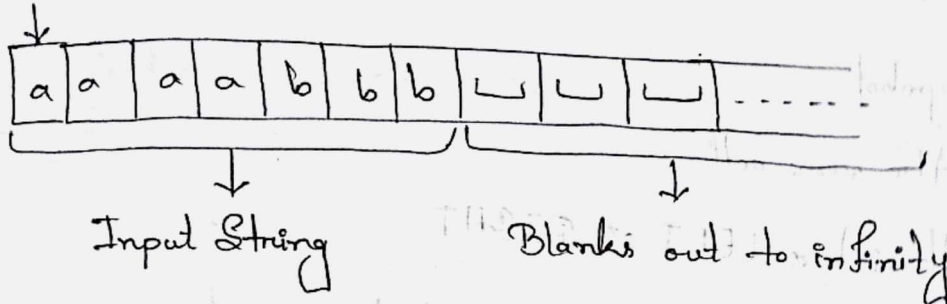
* PDA: $\rightarrow$ The Input String

→ A Stack



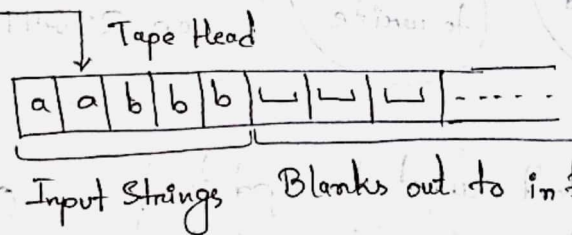
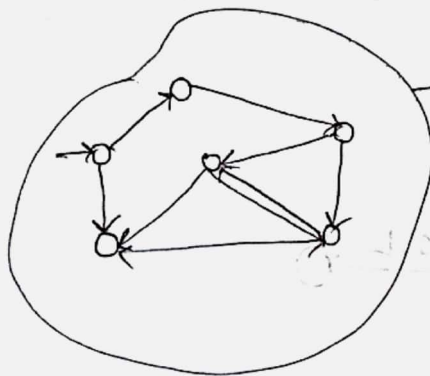
* Initial Configuration:

Tape head



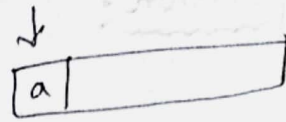
* Operation on the tape:

- Read / Scan symbol below the tape head
- Update / Write a symbol below the tape head
- Move the tape head one step left
- Move the tape head one step right



- The Control Unit similar to FSM or PDA
- The Program
- It's Deterministic

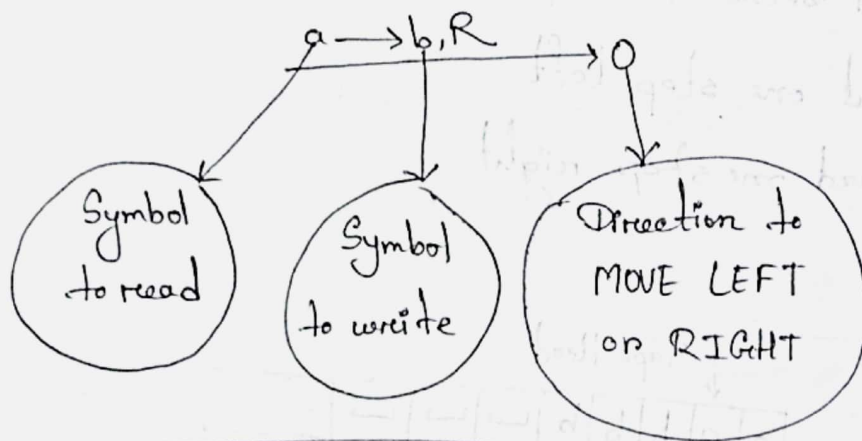
Rules of operation - 1



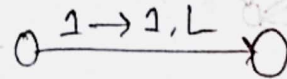
At each step of computation:

- Read the current symbol
- Update (i.e. write) the same cell
- Move exactly one cell either LEFT or RIGHT

If we are at the left end of the tape, and trying to move LEFT, then do not move. Stay at the left end.



If you don't want to update
Just
the cell, write the same symbol



Rules of operation - 02

→ Control is with a sort of FST

→ Initial State

→ Final States (there are two final states)

① The ACCEPT state

② The REJECT state

→ Computation can either

① HALT and ACCEPT

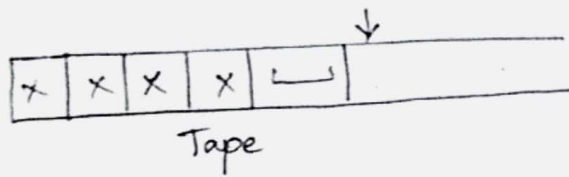
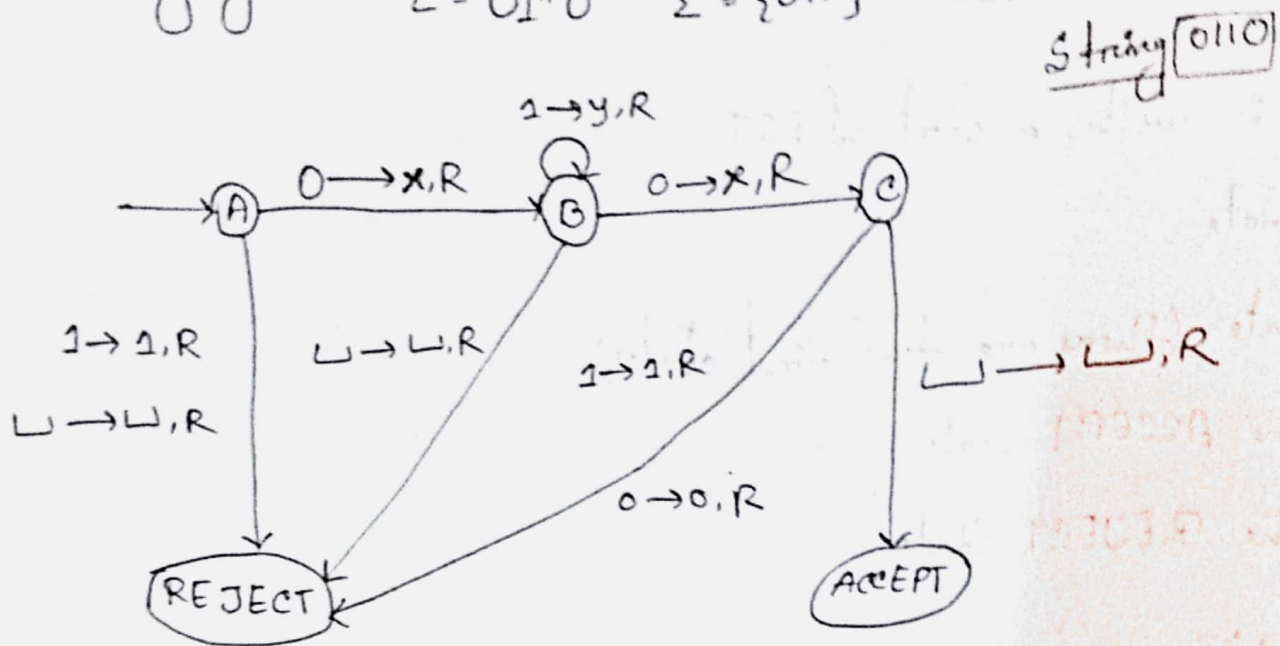
② HALT and REJECT

③ LOOP (the machine fails to HALT)

* Turing's Thesis: Turing's Thesis states that any computation that can be carried out by mechanical means can be performed by some Turing Machine.

* Recursively Enumerable Language (REL): A language $L \subseteq \Sigma^*$ is said to be REL if there exists a Turing Machine that accepts it.

v.v.I
 * Ex - Design a Turing Machine which recognizes the language :- $L = 01^*0$ $\Sigma = \{0,1\}$



* Ex: Design a Turing Machine which recognizes the language $L = 0^N 1^N$

* Algorithm:

① Change "0" to "x"

② Move RIGHT to First '1'

If none :- REJECT

③ Change 1 to "y"

④ Move LEFT to ~~the~~ leftmost "0"

⑤ Repeat the ~~above~~ above steps until no more "0"s.

⑥ Make sure no more "1"s remain

